

NAME: K. Yamini

REG.NO:192572181

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

***CO5:** Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.(BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I _K.Yamini(192572181)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'K. Yamini' with a stylized flourish at the end.

Annexure A

Design Task

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Problem Understanding & Requirement Analysis	10% (1)	/1	/1	/1	/1	/1	/5
System Architecture Design	20% (2)	/2	/2	/2	/2	/2	/10
Use of PySpark & RDD Operations	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Intelligent Agent Integration	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Scalability & Distributed Computing Design	10% (1)	/1	/1	/1	/1	/1	/5
Data Flow & Processing Pipeline	10% (1)	/1	/1	/1	/1	/1	/5
Innovation & Creativity	10% (1)	/1	/1	/1	/1	/1	/5
Clarity, Presentation & Documentation	10% (1)	/1	/1	/1	/1	/1	/5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/50

1. End-to-End System Design – Real-Time Social Media Recommendation System

Introduction

The proposed system analyzes real-time social media data using **PySpark and RDD operations** and generates personalized recommendations. An intelligent agent continuously monitors user activities, analyzes preferences, and recommends relevant content.

Objectives

- Process large volumes of social media data.
- Analyze user likes, comments, shares, and searches.
- Extract useful features from the data.
- Generate personalized recommendations.
- Support real-time processing and scalable computation.

System Architecture

Social Media Data



Data Collection Layer



PySpark / RDD Processing



Data Cleaning & Preprocessing



Feature Extraction



Recommendation Model



Intelligent Recommendation Agent



Personalized Recommendations



User

Data Flow

1. Social media platforms generate posts, likes, comments, and shares.
2. The data is collected continuously.
3. PySpark distributes the data across multiple worker nodes.
4. RDDs clean and transform the raw data.
5. Important features such as user interests and frequently viewed topics are extracted.

6. The recommendation model analyzes these features.
7. The intelligent agent selects suitable content.
8. Recommendations are delivered to users.

PySpark and RDD Operations

RDD transformations such as:

- `map()` – converts raw records into useful formats.
- `filter()` – removes unwanted or invalid data.
- `flatMap()` – extracts multiple items from each record.
- `reduceByKey()` – aggregates user preferences.
- `groupByKey()` – groups information based on users.

RDD actions such as `count()`, `collect()`, and `saveAsTextFile()` can be used to obtain results.

Intelligent Agent

The recommendation agent:

- Monitors user behavior.
- Understands user preferences.
- Selects suitable content.
- Updates recommendations according to new activities.
- Continuously improves the recommendation process.

Scalability and Fault Tolerance

PySpark distributes RDD processing among multiple nodes. If one partition fails, RDDs can be recomputed using their lineage. Therefore, the system can handle large-scale social media data efficiently.

Advantages

- Fast processing.
- Personalized recommendations.
- Handles large datasets.
- Fault tolerant.
- Scalable and distributed.
- Supports real-time decision-making.

Conclusion

Thus, the proposed PySpark-based intelligent recommendation system provides a scalable and efficient solution for analyzing real-time social media data and generating personalized recommendations.

2. Distributed Intelligent Agent System

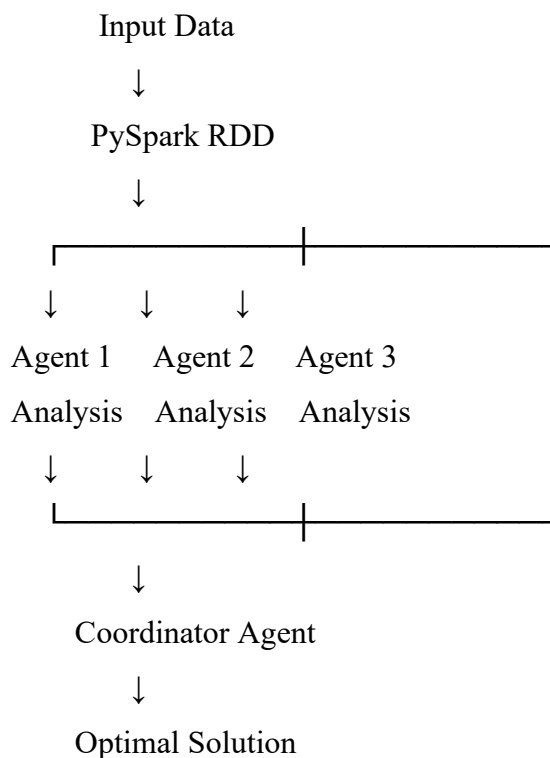
Introduction

A distributed intelligent agent system consists of multiple agents that work together to solve a large-scale optimization problem. **PySpark** performs distributed data processing while agents collaborate to make decisions.

Objectives

- Divide a large optimization problem into smaller tasks.
- Process data in parallel.
- Allow multiple agents to collaborate.
- Improve processing speed and decision-making.

Architecture



Agent Roles

1. Coordinator Agent

- Divides the problem into tasks.
- Assigns tasks to other agents.
- Collects their results.

2. Data Analysis Agent

- Processes and analyzes input data.

3. Optimization Agent

- Finds the best solution for its assigned task.

4. Decision Agent

- Compares the solutions and selects the final result.

PySpark RDD Processing

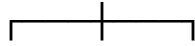
RDDs can divide the dataset into partitions.

For example:

Large Dataset



RDD



P1 P2 P3



A1 A2 A3

Operations such as `map()`, `filter()`, and `reduceByKey()` help process the data in parallel.

Agent Collaboration

Agents communicate their intermediate results to the coordinator agent. The coordinator combines these results and determines the optimal solution.

Scalability

The system can add more worker nodes and agents when the workload increases. PySpark provides parallel processing, while RDD fault tolerance allows failed partitions to be recomputed.

Advantages

- Parallel processing.
- Faster optimization.
- Efficient resource utilization.
- Fault tolerance.
- Scalable architecture.
- Collaborative decision-making.

Conclusion

A distributed intelligent-agent system combined with PySpark provides an effective solution for solving large-scale optimization problems through parallel processing and agent collaboration.

3. Big Data AI Pipeline Design

Introduction

A Big Data AI pipeline processes large datasets through multiple stages such as preprocessing, feature engineering, and model training. PySpark RDDs provide distributed and fault-tolerant processing.

Pipeline Architecture

Raw Big Data



Data Ingestion



RDD Creation



Data Cleaning



Data Preprocessing



Feature Engineering



Distributed Model Training



Model Evaluation



AI Prediction

1. Data Ingestion

Large amounts of data are collected from different sources and loaded into the PySpark environment.

2. RDD Creation

The input data is converted into RDDs so that it can be distributed across multiple worker nodes.

3. Data Preprocessing

RDD transformations are used to:

- Remove missing values.
- Remove duplicate records.
- Filter incorrect data.
- Convert data into suitable formats.

4. Feature Engineering

Important features are extracted from the cleaned data.

Example:

Raw Data



Select useful attributes



Transform attributes



Generate feature vector

5. Model Training

The prepared data is distributed among multiple nodes. AI/ML algorithms can be trained using the processed features.

Important RDD Operations

map() → Transform data

filter() → Select required data

flatMap() → Generate multiple outputs

reduceByKey() → Aggregate data

groupByKey() → Group records

Fault Tolerance

RDDs maintain lineage information. If a partition is lost, Spark can recompute the lost partition from the original transformations.

Scalability

The pipeline distributes data processing across several nodes. More worker nodes can be added when the dataset becomes larger.

Advantages

- Handles big data.
- Parallel processing.
- Fault tolerant.
- Scalable.
- Efficient preprocessing.
- Supports distributed AI model training.

Conclusion

The proposed Big Data AI pipeline provides a reliable and scalable method for converting raw big data into useful AI predictions using PySpark and RDD operations.

4. Real-Time Decision-Making System – Smart Traffic

Introduction

A smart traffic management system can use real-time traffic data, PySpark RDD-based processing, and intelligent agents to make immediate traffic-control decisions.

Objectives

- Monitor traffic continuously.
- Detect congestion.
- Analyze traffic conditions.
- Dynamically control traffic signals.
- Reduce waiting time and congestion.

System Architecture

Traffic Sensors / Cameras



Real-Time Data



PySpark RDD



Data Cleaning & Analysis



Traffic Prediction



Intelligent Traffic Agent



Signal Control Decision



Traffic Lights

Data Processing

Traffic sensors provide:

- Number of vehicles.
- Vehicle speed.
- Road occupancy.
- Traffic density.
- Accident information.

The data is processed using distributed PySpark operations.

RDD Operations

- `filter()` removes invalid sensor readings.

- `map()` converts sensor data into useful information.
- `reduceByKey()` calculates traffic statistics.
- `groupByKey()` groups data according to roads or locations.

Intelligent Agent

The traffic agent continuously observes traffic conditions.

Example:

IF traffic density is HIGH



Increase green-light duration



Reduce congestion

If traffic is low, the agent can reduce unnecessary green-light duration.

Real-Time Decision Process

Observe → Analyze → Decide → Act → Monitor



The agent continuously repeats this process.

Scalability and Fault Tolerance

PySpark distributes traffic data processing across multiple nodes. RDD lineage helps recover lost partitions. Additional nodes can be added when traffic data increases.

Innovation

The system can prioritize:

- Emergency vehicles.
- Accident-prone areas.
- High-density roads.
- Public transportation.

Advantages

- Reduces traffic congestion.
- Faster decision-making.
- Efficient traffic management.
- Real-time monitoring.
- Scalable architecture.

Conclusion

The proposed AI-based smart traffic system combines PySpark, RDD processing, and intelligent agents to provide real-time and adaptive traffic management.

5. Cloud-Based Distributed AI Application

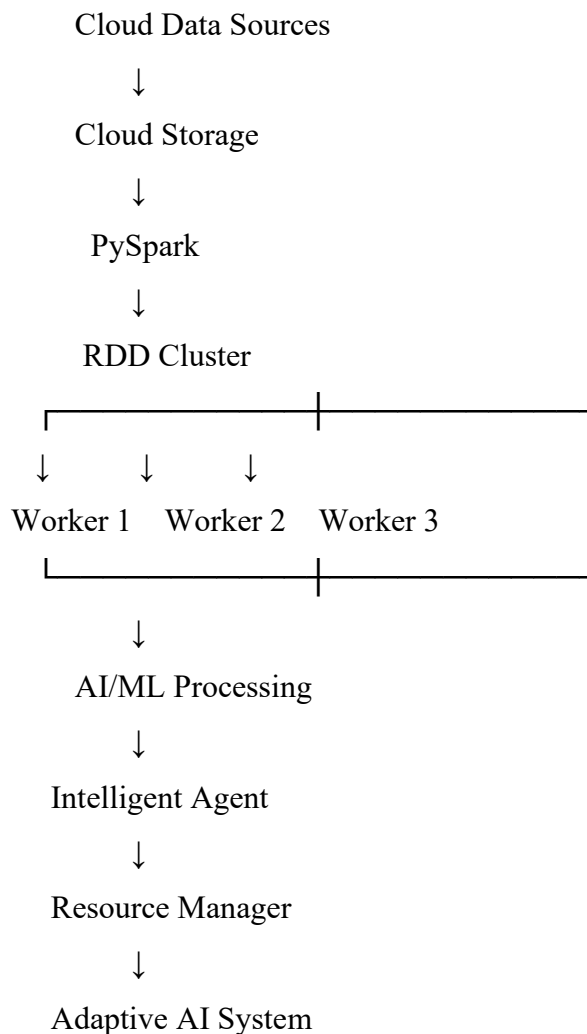
Introduction

A cloud-based distributed AI system combines **PySpark**, **RDDs**, **intelligent agents**, and **cloud computing** to process large-scale data. The system dynamically allocates resources and adapts according to workload.

Objectives

- Process large datasets in the cloud.
- Provide scalable AI computation.
- Dynamically allocate resources.
- Support adaptive learning.
- Provide fault-tolerant processing.

Architecture



Data Processing

1. Data is stored in cloud storage.
2. PySpark loads the data.
3. RDDs divide the data into partitions.
4. Worker nodes process partitions in parallel.

5. AI models analyze the processed data.
6. Intelligent agents monitor system performance.
7. Resources are dynamically adjusted.

Intelligent Agent Integration

The intelligent resource-management agent monitors:

- CPU usage.
- Memory usage.
- Workload.
- Processing time.
- Number of active tasks.

If workload increases, the agent can request additional resources. If workload decreases, unnecessary resources can be released.

RDD Operations

map() → Transform data

filter() → Remove unwanted records

flatMap() → Generate multiple records

reduceByKey() → Aggregate results

Adaptive Learning

The system continuously receives new data and updates its AI model. The intelligent agent monitors model performance and can trigger retraining when required.

Scalability

Cloud resources can be increased or decreased according to workload. PySpark distributes computations among worker nodes, allowing large datasets to be processed efficiently.

Fault Tolerance

RDD lineage allows lost partitions to be recomputed. Distributed cloud infrastructure also allows workloads to be moved to available resources.

Advantages

- Highly scalable.
- Cost-efficient resource utilization.
- Distributed processing.
- Fault tolerant.
- Adaptive learning.
- Suitable for large-scale AI applications.

Conclusion

The cloud-based distributed AI system provides a scalable, intelligent, and adaptive platform for large-scale data analytics by combining PySpark, RDD operations, cloud computing, and

intelligent agents.